PRO2000 - PROGRAMMING

Week 34

Gebremariam Assres
Gebremariam.assres@kristiania.no

Welcome to the course ☺



Today's topics

- Introduction to the course
- Overview of C++
- C++ programming environments

Lesson structure

Week#	Date	Topic	Textbook
34	21.08.26	Introduction to C++	Ch. 1
35	28.08.26	Variables, data types, operators	Ch. 2
36	04.09.26	Conditionals, loops	Ch. 3
37	11.09.26	Functions	Ch. 3, Ch. 7, Ch. 9
38	18.09.26	Arrays, strings, structs	Ch. 3, Ch. 8
39	25.09.26	Pointers, references, memory management	Ch. 15
40	02.10.26	I/O, error handling	Ch. 4, Ch. 9
41	09.10.26	Embedded C++ programming	
*	09.10.26	Mandatory coursework, 1 week duration	
42	16.10.26	Classes, objects, constructors	Ch. 8, Ch. 12
*	16.10.26	Mandatory coursework, handin	
43	23.10.26	Inheritance, polymorphism	Ch. 12
44	30.10.26	Parallel programming	
*	30.10.26	Exam handout, 4 weeks duration	
45	06.11.26	Review	
*	30.11.26?	Exam handin	

Resources

- Textbook:
  - Stroustrup, Bjarne. Programming: principles and practice using C++. Third edition., Hoboken, New Jersey, Addison-Wesley, 2024
- Other references:
  - Viarheichyk, I. (2020). Embedded Programming with Modern C++ Cookbook: Practical recipes to help you build robust and secure embedded applications on Linux. Packt Publishing Ltd.
  - Voss, M., Asenjo, R., & Reinders, J. (2019). Pro TBB: C++ parallel programming with threading building blocks (pp. 25-26). New York: Apress.
  - Malik, D. S. (2006). C++ Programming: From problem analysis to program design. Evaluation, 70(10), 20.
  - Deitel, Paul J, and Harvey M Deitel. C++: How to Program. Ninth edition. Boston: Pearson Education, 2014. Print.
  - Will, Torsten T. C++: The Comprehensive Guide. 1st edition. Bonn: Rheinwerk Publishing, 2025. Print.
  - Deitel, Paul J, and Harvey M Deitel. C++ How to Program: Introducing the New C++ 14 Standard. Tenth edition. Hoboken, New Jersey: Pearson Education, 2017. Print.

Study guide

- We have 12 lessons, always on Fridays.
  - Lectures at 10:15 to 12:00. Exercises at 12:15 to 14:00.

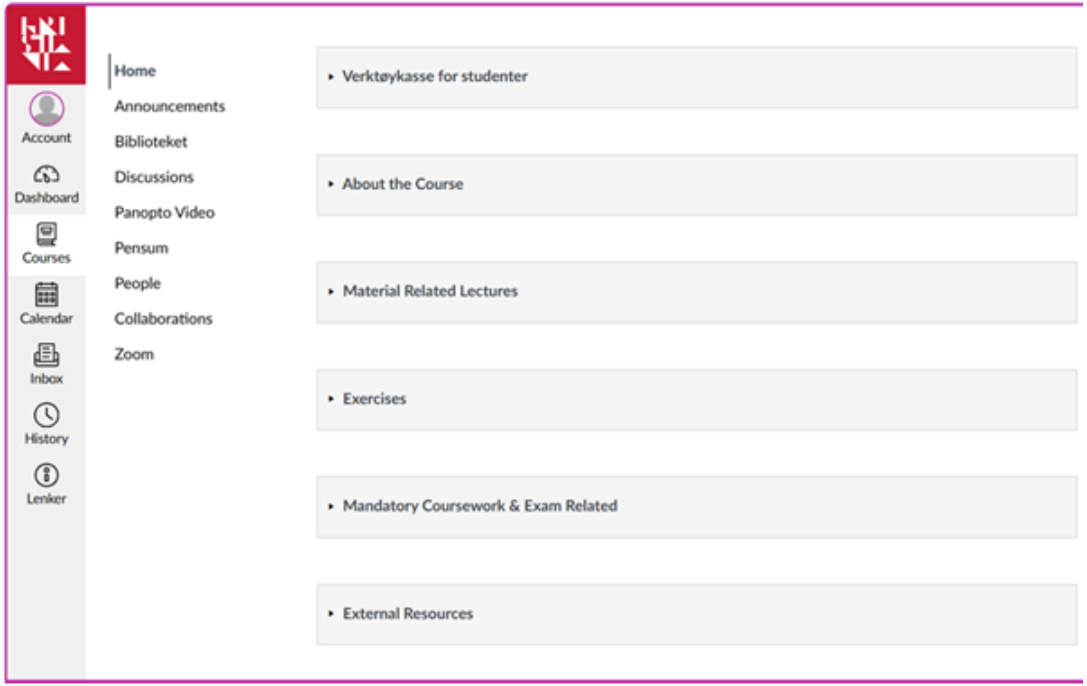
- It takes more than 4 hours a week (the time we share) to learn C++ programming!
  - As a rule of thumb, match every hour with a teacher with an hour on your own.
  - In other words, aim for 8 hours total per week. (4 hours with me, 4 on your own.)

- My suggestion:
  - Start a study-group with some of your fellow students! :-)
  - Agree on some weekdays and timeslots to meet and work together.
  - *For example:* Work 2 hours after the exercises + 2 more hours before next Friday.

- *And remember:* Invite others to join you. Take care of each other! :-)

Canvas – our learning platform

- See Canvas for:
  - Subject description
  - Lesson structure
  - Lecturing material
  - Exercises
  - External resources
- Examination:
  - Portfolio
  - Exam graded A-F



How to participate in my lectures

- A few things I want to mention, so we all get a good experience:

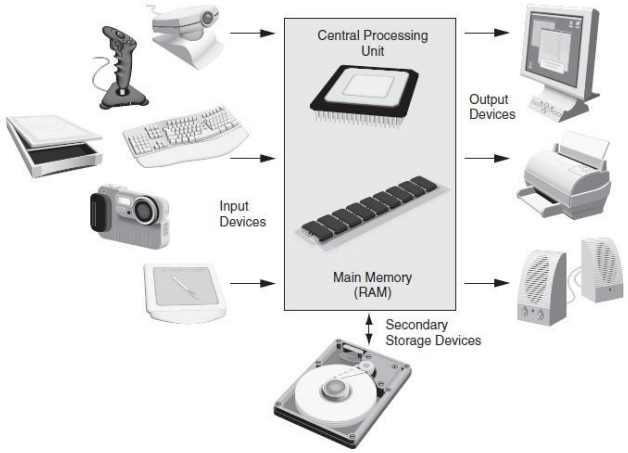
- When you meet up for a lecture, pay attention!
  - Don't check YouTube, ChatGPT, Facebook or do other non-relevant PC-activity. (It's of course ok to do so in the breaks.)
  - Do NOT try to sleep in class!

- *Why these rules?*
  - Out of respect for your fellow students!
  - (And of respect for me – but most importantly, *your fellow students!*)

Overview of C++

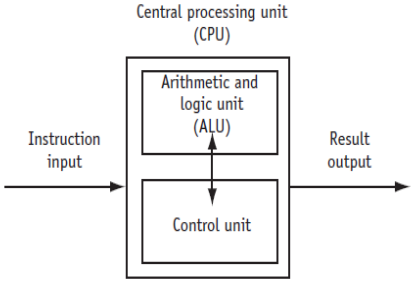
Computer Systems = Hardware + Software

- Hardware – Physical components of a computer and main categories include:
  - Central Processing Unit (CPU)
  - Main memory (RAM)
  - Secondary storage devices
  - Input Devices
  - Output Devices



Computer Systems …

- CPU – Hardware component that runs programs that includes:
  - Control Unit
    - Retrieves and decodes program instructions
    - Coordinates computer operations
  - Arithmetic & Logic Unit (ALU)
    - Performs mathematical operations



Computer Systems …

- The CPU's role in running a program:
  - Iterates through fetch, decode and execute
  - Fetch: get the next program instruction from main memory
  - Decode: interpret the instruction and generate a signal
  - Execute: route the signal to the appropriate component to perform an operation

Computer Systems …

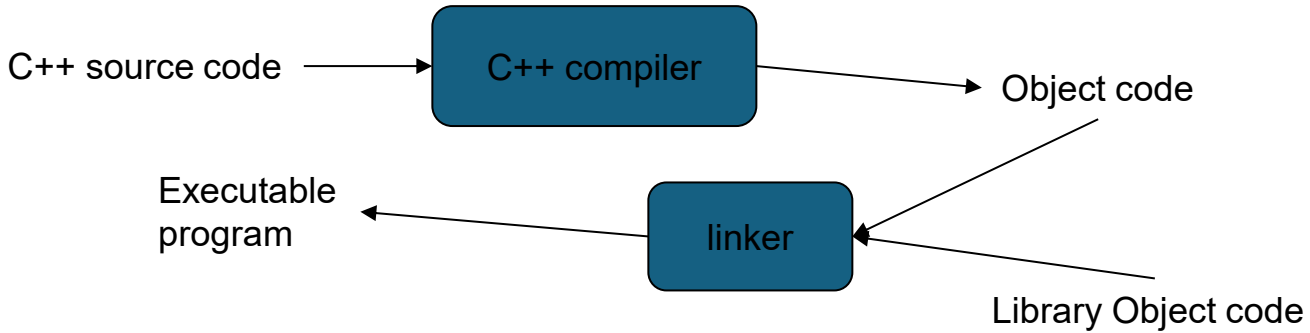
- System software - programs that manage the computer hardware and the programs that run on the computer
  - Operating Systems
    - Controls operation of computer
    - Manages connected devices
    - Runs programs
  - Utility Programs
    - Support programs that enhance computer operations
    - Examples: anti-virus software, data backup, data compression
  - Software development tools
    - Used by programmers to create software
    - Examples: compilers, integrated development environments (IDEs)

Computer Systems …

- Definitions:
  - Program is a set of instructions directing a computer to perform a task
  - Programming Language a language used to write programs
  - Algorithm - a set of steps to perform a task or to solve a problem. Order is important so steps must be performed sequentially
  - Types of languages:
    - Low-level - used for communication with computer hardware directly.
    - High-level - closer to human language

Computer Systems …

- Compilation and linking
  - You write C++ source code - Source code is (in principle) human readable
  - The compiler translates what you wrote into object code (sometimes called machine code which is simple enough for a computer to "understand")
  - The linker links your code to system code needed to execute (E.g., input/output libraries, operating system code, and windowing code)
  - The result is an executable program (E.g., a **.exe** file on windows or an **a.out** file on Unix)



Computer Systems = Hardware + Software

- The preprocessor

`#include` *Inserts file contents verbatim*

```
#include <vector>
#include "mylib.h"
```

`#define` *Text substitution macros*

```
#define MAX_SIZE 256
#define SQ(x) ((x)*(x))
```

`#ifdef` *Conditional compilation*

```
#ifdef DEBUG
  log(msg);
#endif
```

`#pragma` *Compiler-specific hints*

```
#pragma once
#pragma pack(1)
```

Before Preprocessing

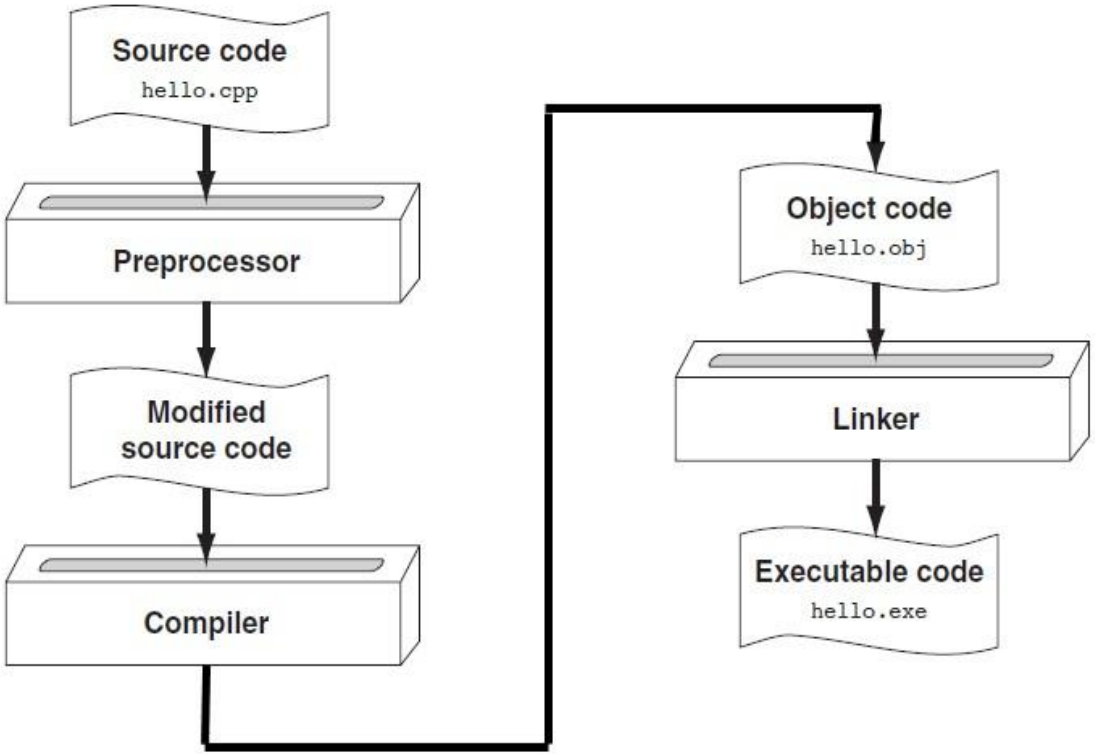
```
#define PI 3.14159
float area = PI * r * r;
```

After Preprocessing

```
// #define removed (macro replaced)
float area = 3.14159 * r * r;
```

Modern C++ Tip: Prefer constexpr over #define for type safety.

Computer Systems = Hardware + Software



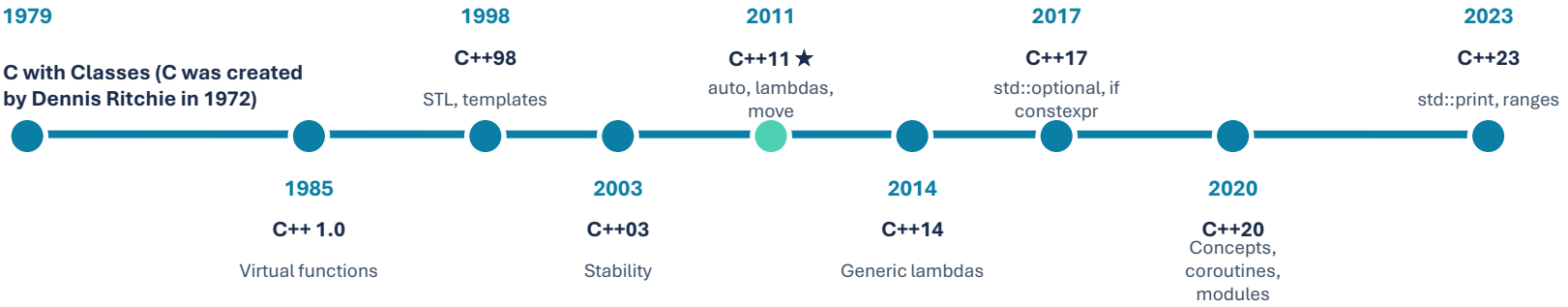
C++ programming

- A programming language allows to express your ideas in code
- You can't learn to program without a programming language
- C++ is the programming language that most directly allows you to express ideas from the largest number of application areas
- C++ is the most widely used language in engineering areas (http://www.stroustrup.com/applications.html)

C++ programming …

- C++ is defined by an ISO standard
  - And that standard is almost universally accepted
  - The most recent officially published C++ standard is C++23 (ISO/IEC 14882:2024)
- C++ is available on almost all kinds of computers
- Programming concepts that you learn using C++ can be used fairly directly in other languages
  - Including C, Java, C#, and (less directly) Fortran

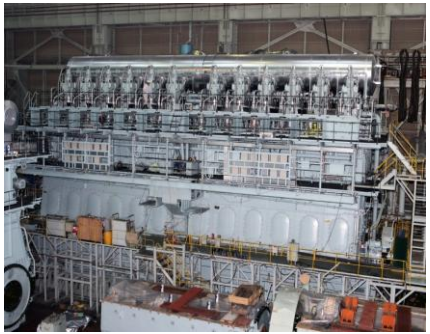
C++ programming ...



C++ programming …

- Our civilization runs on software
  - Most engineering activities involve software
- Note: most programs do not run on things that look like a PC
  - a screen, a keyboard, a box under the table
- There's a lot more to computing than games, word processing, browsing, and spreadsheets!

Ships



Design
Construction
Management



Monitoring
Engine
Hull design
Pumps

Aircraft



Communication
Control
Display



Signal processing
“Gadget” control
Monitoring

Phones



Voice quality
User interfaces
Billing
Mobility



Switching
Reliability
Provisioning
Images

C++ programming ...



General-Purpose

Multi-paradigm: procedural, object-oriented, generic, and functional styles.



High Performance

Zero-overhead abstractions, deterministic memory, inline assembly support.



Systems-Level

Direct hardware access, bit manipulation, memory-mapped I/O.



Portable

Write once, compile everywhere — from microcontrollers to supercomputers.

Programming environments & first C++ program

C++ programming environments

- Required software
  - Visual Studio Code - Source code editor
  - MinGW-w64 - C++ Compiler (g++)
  - gdb – gnu debugger
  - Microsoft C/C++ Extension - IntelliSense and Debugging
  - Read the “Setup Guide VS Code for C++ with MinGW g++, gdb on Windows”

C++ programming environments …

- The setup guide enables to:
  - Install Visual Studio Code
  - Install MinGW-w64 (g++ and gdb)
  - Configure Visual Studio Code
  - Build, Debug and Troubleshooting

A first program- Hello, World!

- Common elements of a program
  - Key Words
  - Programmer-Defined Identifiers
  - Operators
  - Punctuation
  - Syntax- set of rules that defines how symbols, keywords, and punctuation must be ordered to write valid code.

A first program- Hello, World!

- "Hello world" is a very important program
  - Its purpose is to help you get used to your tools
    - Compiler
    - Program development environment
    - Program execution environment
  - Type in the program carefully
    - After you get it to work, please make a few mistakes to see how the tools respond.
    - For example,
      - Forget the header
      - Forget to terminate the string
      - Misspell **return** (e.g., **retrun**)
      - Forget a semicolon
      - Forget **{** or **}**
      - **…**

A first program- Hello, World!

```cpp
// Include the standard I/O stream library
#include <iostream>

// using directive — avoids std:: prefix
using namespace std;

// Entry point — program starts here
int main ) {
    cout << "Hello, World!" << endl;
    return 0;
}
```

#include

Preprocessor directive. Inserts iostream header before compilation.

using

Brings std namespace into scope. Avoid in headers — causes namespace pollution.

int main()

OS calls main() to start your program. Returns int (0 = success).

cout

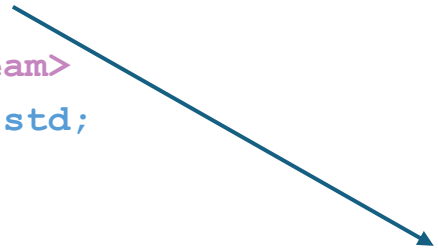
std::cout is the standard output stream object (console).

endl

Flushes buffer and inserts newline. Use '\n' for performance.

A first program- Hello, World!

```
// myHeaders.h
#include <iostream>
using namespace std;
```



```
// main.cpp

#include <myHeaders.h>
int main() {
    cout << "Hello, World!" << endl;
    return 0;
}
```

A first program- Hello, World!

Statement	Purpose
`// My first C++ program`	comment
`#include <iostream>`	preprocessor directive
`using namespace std;`	which namespace to use
`int main()`	beginning of function named main
`{`	beginning of block for main
`    cout << "Hello, World!";`	output statement
`    return 0;`	send 0 back to the operating system
`}`	end of block for main

Up next: Exercises

- Now: 2 hours (12:15 to 14:00) of exercises (found on Canvas).

- Short exercise summary:
  - *Write, compile, and debug a C++ program from scratch*
  - (*If you haven't already:* Install VS Code, Install g++, Install gdb!)

- Remember:
  - You should do approximately 4 more hours of the course (in a study group or alone) *before* our next session together. :-)